

Implementação do circuito quântico correspondente a uma matriz unitária 8×8 por Decomposição QR via Rotações de Givens

PARTE 1 — Resolvendo o caso base

Nessa abordagem, a matriz global U (8×8) atua sobre um sistema de 3 qubits, mas sua ação não trivial está restrita a um subespaço bidimensional específico. Para realizar essa operação utilizando apenas portas quânticas elementares de 1 e 2 qubits, aplicamos o método do Código de Gray para mapear a transição entre os estados da base computacional.

Assim, considere a seguinte matriz de transformação U :

$$U = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & a & 0 & 0 & c \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & b & 0 & 0 & d \end{bmatrix},$$

em que a , b , c e d são números complexos tais que

$$\tilde{U} = \begin{bmatrix} a & c \\ b & d \end{bmatrix}$$

é uma matriz 2×2 unitária.

Primeiramente, é fácil notar que U se comporta como a identidade I para a maioria dos estados da base, exceto para o espaço gerado pelos estados $|100\rangle$ (linha/coluna 4, indexando a partir de 0) e $|111\rangle$ (linha/coluna 7).

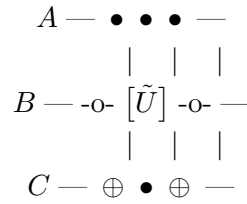
$$U|100\rangle = a|100\rangle + b|111\rangle$$

$$U|111\rangle = c|100\rangle + d|111\rangle$$

Os estados $|100\rangle$ e $|111\rangle$ diferem em 2 bits (os dois últimos qubits). A sequência de transição alterando apenas 1 bit por etapa é (Código de Gray equivalente):

A	B	C	
1	0	0	(Estado Inicial: $ 100\rangle$)
1	0	1	(Altera o 3º bit)
1	1	1	(Altera o 2º bit: $ 111\rangle$)

O circuito correspondente a essa sequência de transições, isolando o subespaço de interesse por meio de portas controladas, é representado esquematicamente a seguir:



PARTE 2 — Decomposição por Rotações de Givens

O foco desta seção é a implementação do circuito quântico correspondente ao cenário em que a transformação unitária $\tilde{U}$ possui dimensão 8×8 , coincidindo com a dimensão total do sistema de 3 qubits. Nesse modelo, a matriz $\tilde{U}$ opera de forma global e incondicional sobre todo o espaço de estados da base computacional (de $|000\rangle$ a $|111\rangle$), sem estar restrita a um subespaço bidimensional específico. Consequentemente, elimina-se a necessidade de usar sequências de Código de Gray ou portas de controle restritas para isolar estados. Para sintetizar essa operação complexa em hardwares quânticos reais, aplica-se o método de decomposição de matrizes unitárias (como a decomposição Coseno-Seno), traduzindo a matriz 8×8 em uma sequência direta de portas quânticas elementares de 1 e 2 qubits que atuam simultaneamente sobre os 3 qubits do sistema.

Vamos fazer isso.

Função de Decomposição em Matrizes 2×2

A função `decompose_two_level_with_indices` abaixo realiza a fatoração de uma matriz unitária complexa de dimensão 8×8 em até 28 submatrizes de 2 níveis, utilizando o princípio das Rotações de Givens aplicado à decomposição QR. O algoritmo inicializa extraindo a dimensão da matriz de entrada U e criando uma cópia mutável `U_curr` configurada com suporte a números complexos. Em seguida, dois laços encadeados percorrem a matriz: o laço externo avança pelas colunas da esquerda para a direita, enquanto o laço interno percorre as linhas de baixo para cima, identificando pares de elementos adjacentes (a, b) localizados na linha âncora $(i - 1)$ e na linha alvo (i) .

Para cada par onde o elemento b é numericamente relevante (superior ao limite de tolerância 10^{-10}), calcula-se a norma $r = \sqrt{|a|^2 + |b|^2}$ e os coeficientes de rotação $c = a/r$ e $s = b/r$. Esses coeficientes constroem uma submatriz unitária 2×2 projetada para zerar o elemento b . Como o processo de eliminação zera a parte inferior da matriz para reduzi-la à identidade, a operação original é reconstruída pela aplicação dos operadores inversos. Por essa razão, a função armazena o operador adjunto conjugado (`u_2x2.conj().T`) junto com os índices de estado $(i - 1, i)$ em uma lista de decomposições.

Por fim, a submatriz 2×2 é incorporada a uma matriz identidade de dimensão 8×8 chamada `V_sub`, posicionando os coeficientes exatamente nas linhas e colunas dos índices ativos. A multiplicação matricial à esquerda `V_sub @ U_curr` atualiza a matriz do sistema, zerando sequencialmente o elemento selecionado e avançando até que a fatoração de todas as 28 etapas de 2 níveis seja concluída.

```

1 import numpy as np
2 from qiskit import QuantumCircuit
3 from qiskit.circuit.library import UnitaryGate
4 from scipy.stats import unitary_group
5
6 # --- Algoritmo de Decomposicao em 2 Niveis (Retorna submatrizes e seus
   indices) ---
7 def decompose_two_level_with_indices(U):
8     N = U.shape[0]
9     decompositions = []

```

```

10     U_curr = U.copy().astype(complex)
11
12     for j in range(N - 1):
13         for i in range(N - 1, j, -1):
14             a = U_curr[i - 1, j]
15             b = U_curr[i, j]
16
17             if np.abs(b) > 1e-10:
18                 r = np.sqrt(np.abs(a)**2 + np.abs(b)**2)
19                 c = a / r
20                 s = b / r
21
22                 # Submatriz 2x2 de rotacao
23                 u_2x2 = np.array([[np.conj(c), np.conj(s)],
24                                   [-s, c]])
25
26                 # Registra a submatriz e os indices i-1 e i onde ela
27                 # atua
28                 decompositions.append({
29                     'u_2x2': u_2x2.conj().T,
30                     'state1': i - 1,
31                     'state2': i
32                 })
33
34                 # Matriz 8x8 de 2 niveis para atualizar U_curr
35                 V_sub = np.eye(N, dtype=complex)
36                 V_sub[i-1, i-1] = np.conj(c)
37                 V_sub[i-1, i] = np.conj(s)
38                 V_sub[i, i-1] = -s
39                 V_sub[i, i] = c
40
41                 U_curr = V_sub @ U_curr
42
43     # Adiciona elementos da diagonal restante, se houver
44     return decompositions

```

Listing 1: Algoritmo de decomposição em dois níveis (Rotações de Givens)

A matriz $U_{i,j}$ representa a extensão de um operador de 2 níveis para o espaço de Hilbert total de dimensão $N \times N$ (no caso, 8×8). Ou seja, a submatriz 2×2 inserida nos cruzamentos das linhas e colunas $i - 1$ e i é a própria rotação de 2 níveis $\tilde{U}$. Ela é responsável por realizar a transformação unitária e zerar o elemento alvo durante a decomposição.

$$U_{i,j} = \begin{bmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & u_{11} & \cdots & u_{12} & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & u_{21} & \cdots & u_{22} & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{bmatrix}$$

Implementação do Código de Gray em cada Matriz e Construção Final do Circuito

O código é dividido em quatro partes principais:

- **Geração do Caminho via Código de Gray (`get_gray_code_path`):** Para atuar entre dois estados arbitrários (por exemplo, de $|000\rangle$ a $|111\rangle$), não é possível mudar múltiplos bits de uma só vez com portas lógicas simples. Esta função cria uma sequência de estados binários onde cada passo difere do anterior por apenas um único bit (propriedade do Código de Gray), construindo um caminho seguro de transição.
- **Mapeamento e Aplicação da Porta de 2 Níveis (`apply_two_level_gate`):** Conduz o sistema através do caminho de Gray aplicando portas Toffoli (CCX) precedidas e sucedidas por inversores X. Isso altera a base computacional para que os dois estados desejados difiram em exatamente 1 bit final.
- **Aplicação da Porta Controlada ($\tilde{U}$):** Com o subespaço isolado, a submatriz 2×2 é aplicada como uma porta unitária controlada por 2 qubits. Portas X adicionais adaptam os controles para atuarem quando os qubits de controle estiverem no estado $|0\rangle$ ou $|1\rangle$, conforme o estado alvo.
- **Descomputação (Uncomputation):** Executa o caminho de Gray em ordem inversa. Essa etapa desfaz as trocas temporárias de estados, garantindo que nenhum qubit auxiliar permaneça alterado e restaurando a integridade dos estados fora do subespaço.

Pipeline Principal (`build_full_unitary_circuit`): Recebe a matriz 8×8 de entrada, invoca a decomposição para obter as submatrizes de 2 níveis e inicializa um circuito quântico de 3 qubits. Em seguida, itera sobre cada submatriz aplicando `apply_two_level_gate` e adiciona uma barreira (`qc.barrier()`) para manter a separação física e evitar fusões indesejadas pelo compilador.

Execução e Exibição: Gera uma matriz unitária 8×8 aleatória, constrói o circuito com as 28 etapas de submatrizes e desenha a estrutura resultante no terminal.

```

1  # --- 2. Funcao de Construção da Etapa de Código de Gray ---
2  def get_gray_code_path(state1_int, state2_int, num_qubits=3):
3      b1 = format(state1_int, f'0{num_qubits}b')
4      b2 = format(state2_int, f'0{num_qubits}b')
5      current, target = list(map(int, b1)), list(map(int, b2))
6      path = [list(current)]
7      for k in range(num_qubits):
8          if current[k] != target[k]:
9              current[k] = target[k]
10             path.append(list(current))
11     return path
12
13 def apply_two_level_gate(qc, u_2x2, state1_int, state2_int):
14     path = get_gray_code_path(state1_int, state2_int)
15
16     # Permutacao inicial
17     for k in range(len(path) - 2):
18         s_from, s_to = path[k], path[k+1]
19         diff_idx = [i for i in range(3) if s_from[i] != s_to[i]][0]
20         control_indices = [i for i in range(3) if i != diff_idx]
21
22         for c in control_indices:
23             if s_from[c] == 0: qc.x(c)
24         qc.ccx(control_indices[0], control_indices[1], diff_idx)
25         for c in control_indices:
26             if s_from[c] == 0: qc.x(c)
27
28     # Aplicacao da porta U_til 2x2 controlada
29     target_state, final_state = path[-2], path[-1]

```

```

30     target_qubit = [i for i in range(3) if target_state[i] !=
31                     final_state[i]][0]
32     control_qubits = [i for i in range(3) if i != target_qubit]
33     u_gate = UnitaryGate(u_2x2, label=r"Util").control(2)
34
35     for c in control_qubits:
36         if target_state[c] == 0: qc.x(c)
37     qc.append(u_gate, control_qubits + [target_qubit])
38     for c in control_qubits:
39         if target_state[c] == 0: qc.x(c)
40
41     # Uncomputation
42     for k in reversed(range(len(path) - 2)):
43         s_from, s_to = path[k], path[k+1]
44         diff_idx = [i for i in range(3) if s_from[i] != s_to[i]][0]
45         control_indices = [i for i in range(3) if i != diff_idx]
46
47         for c in control_indices:
48             if s_from[c] == 0: qc.x(c)
49         qc.ccx(control_indices[0], control_indices[1], diff_idx)
50         for c in control_indices:
51             if s_from[c] == 0: qc.x(c)
52
53     # --- 3. Pipeline Principal de Integracao ---
54     def build_full_unitary_circuit(U_8x8):
55         # Fatora a matriz 8x8 nas etapas de 2 niveis
56         decompositions = decompose_two_level_with_indices(U_8x8)
57
58         qc = QuantumCircuit(3)
59
60         # Aplica cada uma das matrizes de 2 niveis sequencialmente
61         for step in decompositions:
62             apply_two_level_gate(
63                 qc,
64                 u_2x2=step['u_2x2'],
65                 state1_int=step['state1'],
66                 state2_int=step['state2']
67             )
68             qc.barrier() # Separa as etapas visualmente no circuito
69
70         return qc, len(decompositions)
71
72     # --- Exemplo de Execucao ---
73     U_8x8 = unitary_group.rvs(8) # Gera matriz 8x8 aleatoria
74     circuit, num_gates = build_full_unitary_circuit(U_8x8)
75
76     print(circuit.draw('text'))

```

Listing 2: Construção do circuito via Código de Gray e pipeline principal

Custo em Portas

No total, foram utilizadas aproximadamente 216 portas CNOT (8 portas Toffoli, 28 portas controlled- $\tilde{U}$ e 98 portas X de 1 qubit). De acordo com Krol e Al-Ars (2024), a construção ideal da Decomposição por Rotações de Givens para 3 qubits totaliza, no máximo, 192 portas CNOT (Improved QR decomp. — $n \cdot 4^n$).